



Universidad ORT Uruguay
Facultad de Ingeniería

OBLIGATORIO
ESTRUCTURA DE DATOS Y ALGORITMOS 2

Mateo Ferreira (282200)
Matías Piñeyro (271362)

2025

Ejercicios

Ejercicio 1

En este ejercicio, lo que hicimos fue leer los N emails y usamos el OpenHashMap para contar cuántas veces aparece cada uno. Si un email aparece por segunda vez, lo marcamos como repetido y al final calculamos cuántos son únicos a partir de la resta de los repetidos y el total de emails ingresados (N). Y luego lo imprimimos por pantalla.

Al realizar este ejercicio, tuvimos una serie de cosas que nos trancaron un poco. La primera desde el lado de la lógica, estábamos pensando en resolver el ejercicio contando de una los mails que son únicos, lo que nos estaba complicando hasta que en clase nos aconsejaron más ir por el lado de contar los repetidos primero y con eso trabajar para obtener los únicos. De esa manera se nos hizo más fácil resolverlo, y luego la otra dificultad que se nos presentó fue un problema de tipos que no estábamos asociando bien lo que tomaba una función del tad Open HashMap con lo que estaba recibiendo la misma en el código del ejercicio.

La justificación de que su orden temporal sea $O(N)$ es la siguiente:

Las 3 operaciones que utilizo del OpenHashMap son contains(email) con $O(1)$, get(email) con $O(1)$, y set(email, x) con $O(1)$ y tengo N correos para trabajar, por ende $N * O(1) = O(N)$

Muchos de los problemas los consultamos utilizando chat GPT de la página <https://chatgpt.com/> y cuando nos enfrentamos a algún error a la hora de completar el TAD de OpenHashMap también hicimos uso de la herramienta para consultar los errores. Hicimos uso de TADs provistos por el profesor Federico Vilenksy como el de LinkedList, List, etc.

Ejercicio 2

Para el ejercicio 2 fue más el problema de hacer el TAD de closedHashMap que otra cosa, la resolución del problema fue pedir una palabra y una consulta y cargar la palabra N veces a un diccionario (closedHashMap), luego planteamos otro for y le preguntamos mediante el método contains con la consulta, si la consulta estaba en el diccionario devuelvo 1 y si no está devuelvo 0.

Para este ejercicio, en primer lugar tuvimos un poco más de dificultad para completar el TAD ya que nos costó un poco entender el funcionamiento del doble hashing. Entre otros el tema de por qué en la función de hash no se puede usar cualquier número y en clase utilizamos los números primos o por qué debemos hacer que la función de hash 2 devuelva número impar asegurando que si es necesario recorra todos las posiciones sin entrar en ciclo. Otro problema al que nos enfrentamos fue que nos daba el error de "Segmentation" y lo que estaba pasando es que no habíamos aplicado el valor absoluto al resultado de la función de hash en el método contains.

La justificación de su orden temporal es la siguiente:

Tenemos las operaciones set de $O(1)$ y contains también de $O(1)$ (Se cumple lo que pide la letra) y luego tengo N palabras que quiero insertar al diccionario + M consultas que voy a hacerle luego. Entonces el orden temporal sería el siguiente $O(N + M)$.

Hicimos uso de chat GPT de la página <https://chatgpt.com/> para consultar errores y de Geeks for Geeks (<https://www.geeksforgeeks.org/>) para tener una referencia. De nuevo hicimos uso de TADs provistos por el profesor, como LinkedList, Map, entre otros.

Ejercicio 3

En este ejercicio pensamos en utilizar una estructura de array e implementamos una función de heapify, donde construimos un heap y luego lo sorteamos como pide la letra (ordenar de menor a mayor), así cumpliendo con lo que pide la letra.

La justificación de su orden temporal es la siguiente:

Nosotros tenemos N elementos que queremos ordenar, en la función `sortH()` creamos el heap mediante la función `heapify()`, esta última tiene orden temporal $O(N)$, luego extraemos los elementos del heap haciendo un swap para luego armarlo nuevamente ejecutandolo $n - 1$ veces, entonces ahí su orden temporal sería $O(N * \log N)$, al final nos queda $O(N) + O(N * \log N) = O(N * \log N)$.

Para este ejercicio utilizamos la página <https://www.geeksforgeeks.org/> como referencia para entender el funcionamiento del heapify explicado en clase.

Ejercicio 4

A partir de las restricciones de la letra, planteamos un array de costos inicializado en `INT_MAX` (`INF`), un array de visitados inicializado en `false` y el costo del origen igual a 0. A partir de ahí aplicamos Dijkstra que consta de extraer el nodo de menor costo, marcarlo como visitado y actualizar los costos de sus adyacentes si encontramos un camino más corto. De esta manera, cumplimos con la complejidad requerida de $O((N(V + E) \log V))$ y resolvimos el problema.

La justificación del orden temporal es la siguiente:

Dado que se ejecuta el algoritmo de Dijkstra desde N vértices origen, lo justificamos para una sola ejecución. En la función `dijkstra()`, inicializamos dos array de tamaño V , lo cual tiene costo $O(V)$. Luego por cada vértice, se procesan sus adyacentes recorriendo todas las aristas del grafo, lo que lleva a $O(E)$. En este proceso, se realizan operaciones como insertar y extraer elementos sobre el MinHeap, con un costo de $O(\log V)$ cada una. Como a lo sumo se procesan todos los vértices y todas las aristas una vez el costo es como vimos en clase: $O((V+E) \cdot \log V)$. Dado que se aplica para cada vértice (cada origen es uno diferente), el orden final queda: $O(N(V+E) \cdot \log V)$ como dice la letra del ejercicio.

En este caso usamos el código provisto en clase para el TAD ListGraph y usamos el MinHeap implementado con apoyo de lo visto en clase y apoyo de chatgpt para consultar errores <https://chatgpt.com/>.

Ejercicio 5

Para resolver este ejercicio, pensamos en recorrer el grafo usando DFS desde cada vértice no visitado, con el fin de detectar ciclos. Lo que consideramos más importante fue usar un array extra (`enPila[]`) para saber si un vértice está actualmente en la pila de llamadas del DFS. Si durante la recorrida llegamos a un vértice que ya está en la pila, significa que hay un ciclo. Esto nos permitió resolver el ejercicio, respetando la complejidad $O(V+E)$.

La justificación del orden temporal es la siguiente:

El algoritmo recorre todos los vértices y aristas del grafo utilizando DFS. Cada vértice se visita una sola vez y se marcan sus adyacentes también una vez, cumpliendo así con el recorrido de todas las aristas. Por lo tanto, el recorrido completo tiene una complejidad de $O(V+E)$, que corresponde al tiempo que se requiere para procesar todos los vértices (V) y todas las aristas (E) en un grafo representado mediante listas de adyacencia.

En este caso, lo terminamos de resolver luego del parcial por lo que la dificultad fue un poco menor porque ya teníamos bastante estudiado el algoritmo DFS. Además utilizamos el código al igual que en el ejercicio 4 del TAD ListGraph provisto por el profesor en clase.

Resultados

Ejercicio	Resultado
1	Completo
2	Parcial
3	Completo
4	Completo
5	Completo

Aclaración sobre el ejercicio 2, pasan las pruebas 1 y 2 pero la prueba 3 es la que no está dando, por eso colocamos que su resultado es parcial.

- Completo: Corren todas las pruebas.
- Parcial: Corren parcialmente las pruebas.
- En Proceso: Implementado pero no pasa ninguna prueba.
- No Implementado: Ejercicio no implementado o no compila.